B.L.A.S.T. OCR

I benchmarked my own OCR tool, then threw out its engine.

117.8s to 15.3s a page

Swipe for the numbers, including the ones that look bad.

THE REAL PROBLEM

It was never accuracy. It was that nobody would wait.

EasyOCR on CPU took 117.8 seconds a page. A 300-page scanned book is close to ten hours of compute before you see a single line of text. That is not slow. That is unusable.

Measured on my own machine, single stream, no GPU.

WHAT ACTUALLY CHANGED

RapidOCR, running on ONNX Runtime.

Same detection and recognition approach. Different runtime. ONNX Runtime picks up CUDA or DirectML when the hardware is there and drops back to CPU when it isn't, so there is no PyTorch left in the hot path.

Swap documented in ADR 0005, in the repo.

THE NUMBERS

Same corpus. Both engines.

METRIC	EASYOCR TO RAPIDOCR
Mean CER	0.2338 → **0.1916**
CPU latency / page	117.8s → **15.3s**
Reading order score	0.9641 → **0.9758**

14-page gold corpus, CPU only. Raw JSON is committed.

Then I ran 1,000 pages to see if it would leak.

0.0002 MB/page

Growth slope across a 1,000-page streaming run, against a 0.005 MB/page fail threshold. It will not balloon RAM on a long book.

`eval/results/stress_report.json`

WHAT I STILL CAN'T TELL YOU

The gaps, before you find them.

- 15.3s a page is still slow.
- 14 pages is a small corpus.
- No GPU throughput number.
- No table-accuracy score.

All four are logged in the repo as open gaps.

IT'S YOURS

MIT licensed. Runs offline. No API keys.

github.com/Ibrahim-Salman19/OCR

What's the document your OCR stack still can't read? Tell me and I'll point this at it.